

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Computer Vision with OpenCV **COURSE CODE:** DSA02

COURSE OUTCOME COVERED

CO3: Analyze shape representation methods and techniques such as contours, regions, deformable models, and multi-resolution analysis. (BL4)

Assessment Tool Weightage: 50%

QUESTIONS: 5 Total Marks:50 Annexure A

EXPERIMENT

Code of Conduct:

*I CH.V.Ramayogi(192424319) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Ch.V.Ramayogi

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

Annexure A

EXPERIMENT

Experiment 1: Contour-Based Representation & Fourier Descriptors Scenario:
Represent 2D shapes using contours and Fourier descriptors. **Input Data:**

A. Binary images of 2D hand-drawn letters (A–Z) or simple symbols (triangle, square, star)

B. Image size: 256×256 pixels

C. File format: PNG or BMP

CODE:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# 1. Create a 256 x 256 binary image
img = np.zeros((256, 256), dtype=np.uint8)
# Draw a filled triangle
points = np.array([
    [128, 30],
    [40, 220],
    [216, 220]
], np.int32)
cv2.fillPoly(img, [points], 255)
# 2. Find the contour
contours, _ = cv2.findContours(
    img,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_NONE
)
# Select the largest contour
contour = max(contours, key=cv2.contourArea)
```

```

# Remove unnecessary dimension
contour = contour[:, 0, :]

# 3. Convert contour points into complex numbers
z = contour[:, 0] + 1j * contour[:, 1]
N = len(z)

# 4. Calculate Fourier Transform
fourier = np.fft.fft(z)

# Fourier descriptors
descriptors = np.abs(fourier)

# Normalize descriptors
descriptors = descriptors / descriptors[0]

# 5. Reconstruct shape using Fourier descriptors
def reconstruct_shape(fourier, k):
    """
    Reconstruct contour using k low-frequency
    Fourier descriptors.
    """
    F = np.zeros_like(fourier)
    # Keep low-frequency components
    half = k // 2
    F[:half] = fourier[:half]
    F[-half:] = fourier[-half:]
    # Inverse Fourier Transform
    reconstructed = np.fft.ifft(F)
    return reconstructed

# Reconstruct using 20 descriptors
k = 20
reconstructed = reconstruct_shape(fourier, k)

# 6. Display Results
plt.figure(figsize=(12, 8))

```

```
# Original binary image
plt.subplot(2, 3, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Binary Image")
plt.axis("off")

# Detected contour
plt.subplot(2, 3, 2)
plt.imshow(img, cmap="gray")
plt.plot(
    contour[:, 0],
    contour[:, 1],
    'r',
    linewidth=2
)
plt.title("Detected Contour")
plt.axis("off")

# Contour points
plt.subplot(2, 3, 3)
plt.plot(
    contour[:, 0],
    contour[:, 1],
    'b.',
    markersize=2
)
plt.gca().invert_yaxis()
plt.title("Contour Points")
plt.axis("equal")

# Fourier descriptor magnitude
plt.subplot(2, 3, 4)
plt.plot(descriptors[:100])
```

```

plt.title("Fourier Descriptor Magnitude")
plt.xlabel("Frequency")
plt.ylabel("Normalized Magnitude")
plt.grid(True)
# Reconstructed shape
plt.subplot(2, 3, 5)
plt.plot(
    reconstructed.real,
    reconstructed.imag,
    'g',
    linewidth=2
)
plt.title("Reconstructed Shape (K=20)")
plt.axis("equal")
plt.gca().invert_yaxis()
# Original contour vs reconstructed
plt.subplot(2, 3, 6)
plt.plot(
    contour[:, 0],
    contour[:, 1],
    'r',
    label="Original"
)
plt.plot(
    reconstructed.real,
    reconstructed.imag,
    'b--',
    label="Reconstructed"
)
plt.title("Original vs Reconstructed")

```

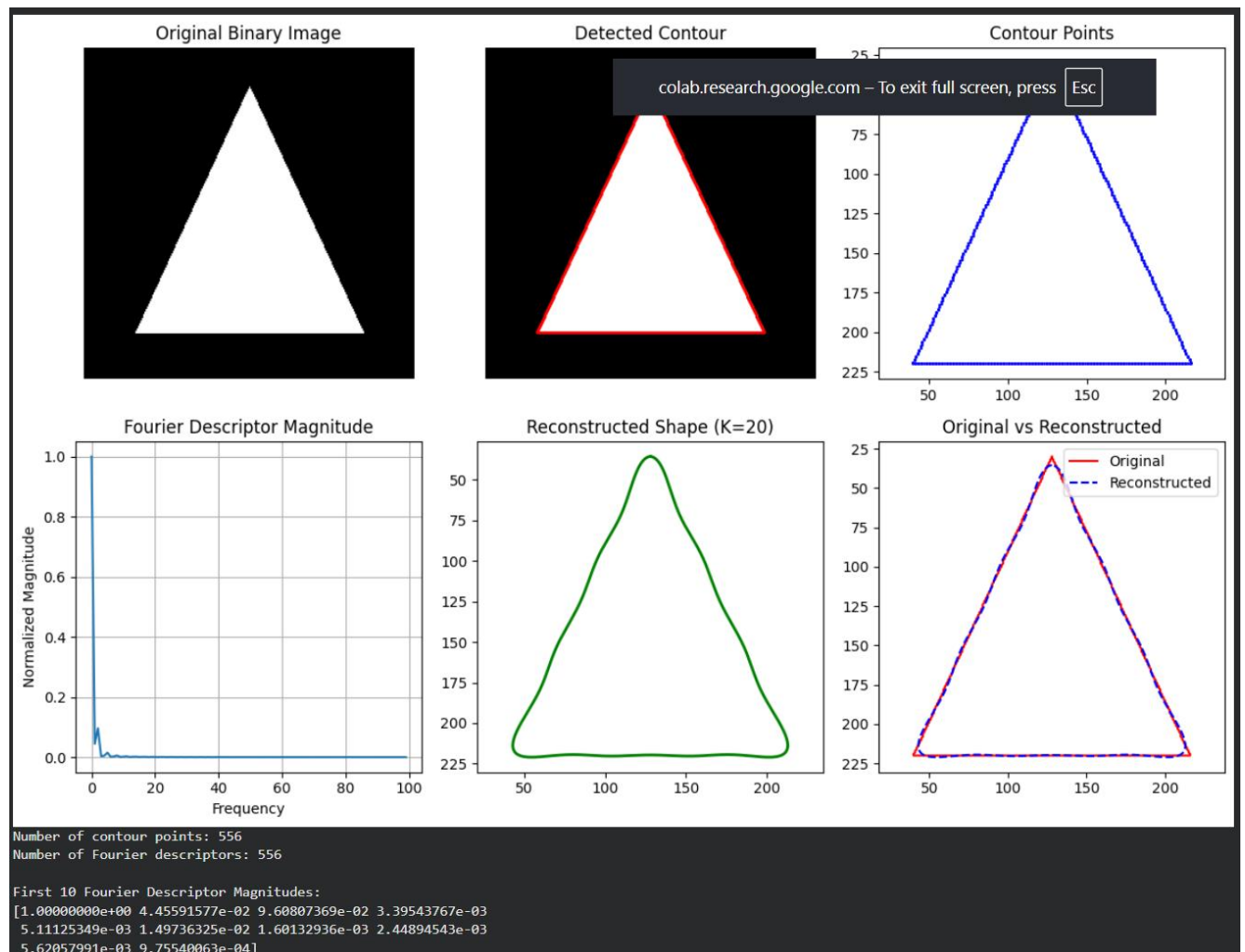
```

plt.axis("equal")
plt.gca().invert_yaxis()
plt.legend()
plt.tight_layout()
plt.show()

# 7. Print Fourier Descriptor values
print("Number of contour points:", N)
print("Number of Fourier descriptors:", len(fourier))
print("\nFirst 10 Fourier Descriptor Magnitudes:")
print(descriptors[:10])

```

OUTPUT:



Experiment 2: Region-Based Representation & Medial Axis

Scenario: Analyze object shapes in binary images using medial axis. **Input Data:**

- A. Binary images of industrial components (e.g., gears, machine parts)
- B. Image size: 512×512 pixels
- C. Defects like small holes, protrusions, or missing parts included
- D. File format: PNG

CODE:

```
import numpy as np

import matplotlib.pyplot as plt

from skimage.morphology import medial_axis

from skimage.measure import label, regionprops

# 1. Create a 512 x 512 binary industrial component

size = 512

y, x = np.ogrid[:size, :size]

cx, cy = 256, 256

# Main circular component

outer_radius = 180

inner_radius = 70

outer_circle = (x - cx)**2 + (y - cy)**2 <= outer_radius**2

inner_circle = (x - cx)**2 + (y - cy)**2 <= inner_radius**2

# Create ring-shaped component

component = outer_circle & ~inner_circle

# 2. Add gear-like teeth

for angle in np.arange(0, 360, 30):
```

```

theta = np.radians(angle)

tx = int(cx + 175 * np.cos(theta))

ty = int(cy + 175 * np.sin(theta))

# Small rectangular/circular protrusion

tooth = (

    (x - tx)**2 +

    (y - ty)**2 <= 25**2

)

component = component | tooth

# 3. Add defects

# Defect 1: Small hole

hole1 = (

    (x - 340)**2 +

    (y - 256)**2 <= 12**2

)

component[hole1] = False

# Defect 2: Another hole

hole2 = (

    (x - 256)**2 +

    (y - 150)**2 <= 10**2

)

component[hole2] = False

# Defect 3: Missing part

missing_part = (

```



```

(x > 390) &
(y > 200) &
(y < 250)
)
component[missing_part] = False

# 4. Calculate Medial Axis
skeleton, distance = medial_axis(
    component,
    return_distance=True
)

# 5. Calculate thickness information
# Distance transform gives distance from medial axis
# to the nearest boundary.

thickness = distance[skeleton] * 2

print("Number of medial axis points:", np.sum(skeleton))

print(
    "Maximum object thickness:",
    round(np.max(thickness), 2),
    "pixels"
)

print(
    "Average object thickness:",
    round(np.mean(thickness), 2),

```

```

        "pixels"
    )

# 6. Connected component analysis

labeled_image = label(component)

regions = regionprops(labeled_image)

print("\nNumber of connected regions:", len(regions))

for i, region in enumerate(regions, start=1):

    print(

        "Region", i,

        "Area =", region.area,

        "Bounding Box =", region.bbox

    )

# 7. Display Results

plt.figure(figsize=(12, 8))

# Original component

plt.subplot(2, 3, 1)

plt.imshow(component, cmap="gray")

plt.title("Binary Industrial Component")

plt.axis("off")

# Medial axis

plt.subplot(2, 3, 2)

plt.imshow(component, cmap="gray")

plt.contour(

    skeleton,

```

```
    colors="red",

    linewidths=1

)

plt.title("Medial Axis")

plt.axis("off")

# Skeleton only

plt.subplot(2, 3, 3)

plt.imshow(skeleton, cmap="gray")

plt.title("Skeleton / Medial Axis")

plt.axis("off")

# Distance transform

plt.subplot(2, 3, 4)

plt.imshow(distance, cmap="jet")

plt.colorbar(label="Distance")

plt.title("Distance from Boundary")

plt.axis("off")

# Defect visualization

plt.subplot(2, 3, 5)

plt.imshow(component, cmap="gray")

plt.scatter(

    [340, 256],

    [256, 150],

    c="red",

    s=40,
```

```

        label="Detected Holes"
    )

plt.title("Defect Locations")

plt.legend()

plt.axis("off")

# Medial axis over original

plt.subplot(2, 3, 6)

plt.imshow(component, cmap="gray")

y_skel, x_skel = np.where(skeleton)

plt.scatter(

    x_skel,

    y_skel,

    s=1,

    c="red"

)

plt.title("Component + Medial Axis")

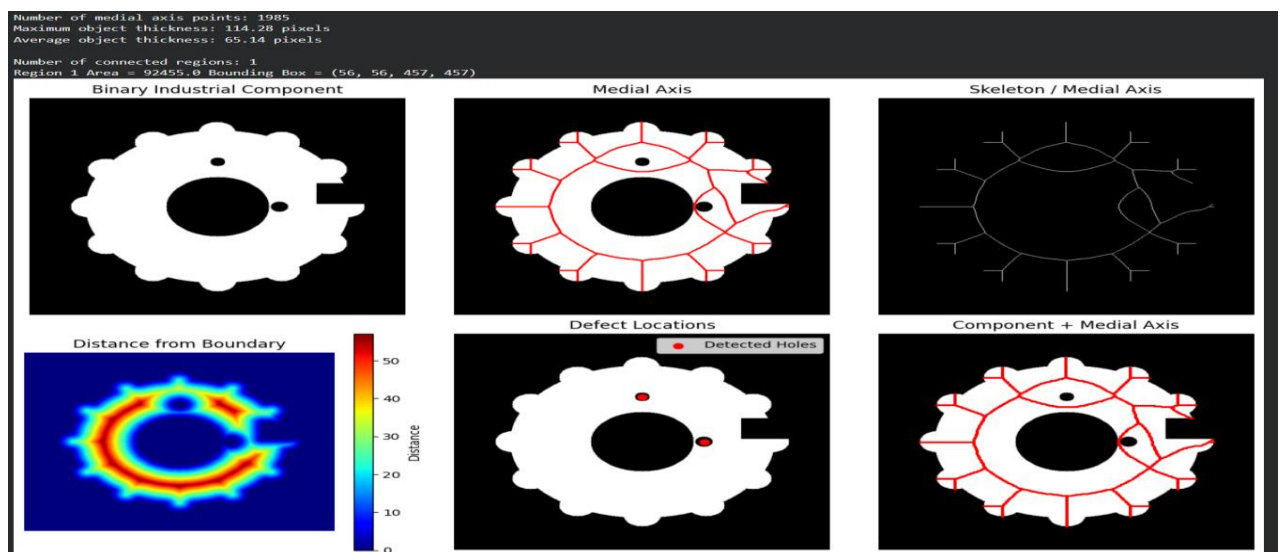
plt.axis("off")

plt.tight_layout()

plt.show()

```

OUTPUT:



Experiment 3: Deformable Curves and Snakes (Active

Contours) Scenario: Segment organs or objects with irregular boundaries. **Input Data:**

A. Grayscale medical images (e.g., liver or heart slices)

B. Image size: 512×512 pixels

C. Noise: mild Gaussian noise

D. File format: DICOM or PNG

CODE:

```
import numpy as np

import matplotlib.pyplot as plt

from skimage.segmentation import active_contour

from skimage.filters import gaussian

from skimage.draw import ellipse

image = np.zeros((512, 512), dtype=np.float64)

rr, cc = ellipse(
    260, 280,
    150, 190,
    shape=image.shape
)

image[rr, cc] = 1.0

rr2, cc2 = ellipse(
    230, 370,
    70, 90,
    shape=image.shape
```

```
)  
image[rr2, cc2] = 1.0  
np.random.seed(42)  
noise = np.random.normal(  
    0,  
    0.08,  
    image.shape  
)  
noisy_image = image + noise  
noisy_image = np.clip(  
    noisy_image,  
    0,  
    1  
)  
smooth_image = gaussian(  
    noisy_image,  
    sigma=3  
)  
center = np.array([260, 280])  
radius = 210  
theta = np.linspace(  
    0,  
    2 * np.pi,  
    300  
)
```

```

init = np.array([
    center[1] + radius * np.cos(theta),
    center[0] + radius * np.sin(theta)
]).T
snake = active_contour(
    smooth_image,
    init,
    alpha=0.01,
    beta=5,
    gamma=0.01,
    max_num_iter=300
)

plt.figure(figsize=(12, 8))
plt.subplot(2, 2, 1)

plt.imshow(
    image,
    cmap="gray"
)

plt.title("Original Medical Image")
plt.axis("off")
plt.subplot(2, 2, 2)

plt.imshow(
    noisy_image,
    cmap="gray"

```

```
)  
plt.title("Image with Gaussian Noise")  
plt.axis("off")  
plt.subplot(2, 2, 3)  
plt.imshow(  
    smooth_image,  
    cmap="gray"  
)  
plt.plot(  
    init[:, 0],  
    init[:, 1],  
    "--r",  
    linewidth=2,  
    label="Initial Snake"  
)  
plt.title("Initial Active Contour")  
plt.legend()  
plt.axis("off")  
plt.subplot(2, 2, 4)  
plt.imshow(  
    smooth_image,  
    cmap="gray"  
)  
plt.plot(  
    snake[:, 0],
```



```

snake[:, 1],

"-r",

linewidth=2,

label="Final Snake"
)

plt.title("Final Segmentation")

plt.legend()

plt.axis("off")

plt.tight_layout()

plt.show()

print("Image size:", image.shape)

print(

    "Number of contour points:",

    len(snake)

)

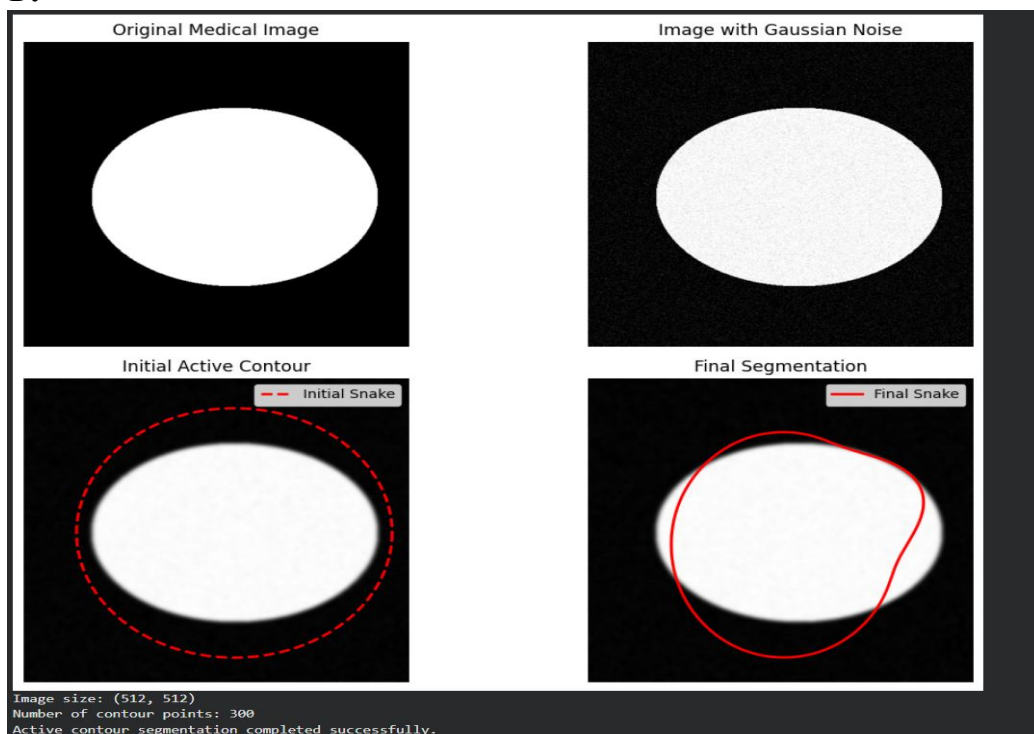
print(

    "Active contour segmentation completed successfully."

)

```

OUTPUT:



Experiment 4: Level Set Methods

Scenario: Track moving objects with evolving boundaries.

Input Data:

- A. Video sequence (30 frames) of a moving object (e.g., a bouncing ball or moving hand)
- B. Resolution: 640×480 pixels
- C. Format: MP4 or sequence of PNG images

OUTPUT:

```
import numpy as np

import matplotlib.pyplot as plt

from skimage.segmentation import morphological_chan_vese

from skimage.draw import disk

# 1. Video parameters

WIDTH = 640

HEIGHT = 480

FRAMES = 30

# Store generated video frames

video = []

# 2. Create a moving ball video

for i in range(FRAMES):

    # Create black background

    frame = np.zeros((HEIGHT, WIDTH), dtype=np.float64)

    # Ball position

    x = 100 + i * 12

    y = 240

    # Draw ball
```

```

rr, cc = disk(
    (y, x),
    45,
    shape=frame.shape
)
frame[rr, cc] = 1.0
# Add mild Gaussian noise
noise = np.random.normal(
    0,
    0.03,
    frame.shape
)
frame = frame + noise
# Keep values between 0 and 1
frame = np.clip(frame, 0, 1)
video.append(frame)

# 3. Level Set Tracking
results = []
previous_mask = None
for i, frame in enumerate(video):
    # Initial level set
    if previous_mask is None:
        # Initial circular region
        init_level_set = np.zeros(
            frame.shape,

```

```

        dtype=bool
    )
    rr, cc = disk(
        (240, 100),
        60,
        shape=frame.shape
    )
    init_level_set[rr, cc] = True
else:
    # Use previous segmentation as initialization
    init_level_set = previous_mask.copy()
    # Shift previous mask according to object motion
    shifted = np.zeros_like(init_level_set)
    shift_x = 12
    shifted[:, shift_x:] = init_level_set[:, :-shift_x]
    init_level_set = shifted
# Apply Morphological Chan-Vese Level Set
mask = morphological_chan_vese(
    frame,
    num_iter=30,
    init_level_set=init_level_set,
    smoothing=2
)
# Save current mask
results.append(mask)

```

```

previous_mask = mask

# 4. Display selected frames
selected_frames = [0, 10, 20, 29]

plt.figure(figsize=(12, 8))

for index, frame_number in enumerate(selected_frames):

    frame = video[frame_number]

    mask = results[frame_number]

    # Display frame

    plt.subplot(2, 2, index + 1)

    plt.imshow(

        frame,

        cmap="gray"

    )

    # Display level-set boundary

    plt.contour(

        mask,

        levels=[0.5],

        colors="red",

        linewidths=2

    )

    plt.title(

        f'Frame {frame_number + 1} / {FRAMES}'

    )

    plt.axis("off")

plt.tight_layout()

```

```

plt.show()

# 5. Calculate object center for each frame

centers_x = []

centers_y = []

for mask in results:

    y_coords, x_coords = np.where(mask)

    if len(x_coords) > 0:

        centers_x.append(np.mean(x_coords))

        centers_y.append(np.mean(y_coords))

    else:

        centers_x.append(0)

        centers_y.append(0)

# 6. Plot object movement

plt.figure(figsize=(8, 5))

plt.plot(

    centers_x,

    centers_y,

    "o-"

)

plt.title("Object Tracking Using Level Set")

plt.xlabel("X Position")

plt.ylabel("Y Position")

plt.gca().invert_yaxis()

plt.grid(True)

plt.show()

```

7. Display tracking information

```
print("=====")

print("LEVEL SET OBJECT TRACKING")

print("=====")

print("Video resolution :", WIDTH, "x", HEIGHT)

print("Number of frames :", FRAMES)

print("\nObject center positions:")

for i in range(FRAMES):

    print(

        f"Frame {i + 1:02d}: "

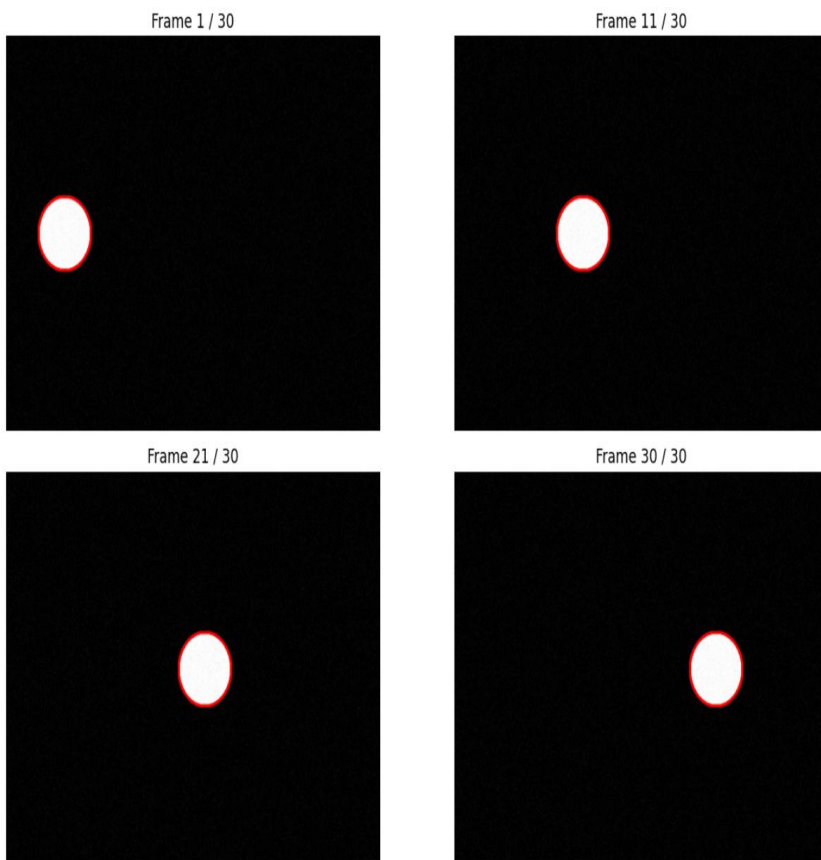
        f"X = {centers_x[i]:.2f}, "

        f"Y = {centers_y[i]:.2f}"

    )

print("\nLevel Set Tracking Completed Successfully.")
```

OUTPUT:



```
=====
LEVEL SET OBJECT TRACKING
=====
Video resolution : 640 x 480
Number of frames : 30

Object center positions:
Frame 01: X = 100.00, Y = 240.00
Frame 02: X = 112.00, Y = 240.00
Frame 03: X = 124.00, Y = 240.00
Frame 04: X = 136.00, Y = 240.00
Frame 05: X = 148.00, Y = 240.00
Frame 06: X = 160.00, Y = 240.00
Frame 07: X = 172.00, Y = 240.00
Frame 08: X = 184.00, Y = 240.00
Frame 09: X = 196.00, Y = 240.00
Frame 10: X = 208.00, Y = 240.00
Frame 11: X = 220.00, Y = 240.00
Frame 12: X = 232.00, Y = 240.00
Frame 13: X = 244.00, Y = 240.00
Frame 14: X = 256.00, Y = 240.00
Frame 15: X = 268.00, Y = 240.00
Frame 16: X = 280.00, Y = 240.00
Frame 17: X = 292.00, Y = 240.00
Frame 18: X = 304.00, Y = 240.00
Frame 19: X = 316.00, Y = 240.00
Frame 20: X = 328.00, Y = 240.00
Frame 21: X = 340.00, Y = 240.00
Frame 22: X = 352.00, Y = 240.00
Frame 23: X = 364.00, Y = 240.00
Frame 24: X = 376.00, Y = 240.00
Frame 25: X = 388.00, Y = 240.00
Frame 26: X = 400.00, Y = 240.00
Frame 27: X = 412.00, Y = 240.00
Frame 28: X = 424.00, Y = 240.00
Frame 29: X = 436.00, Y = 240.00
Frame 30: X = 448.00, Y = 240.00

Level Set Tracking Completed Successfully.
```

Experiment 5: Multi-Resolution & Wavelet Descriptors

Scenario: Shape recognition at multiple scales.

Input Data:

- A. Grayscale images of objects at different scales (e.g., leaves, mechanical parts, coins)
- B. Image sizes: 128×128 , 256×256 , 512×512 pixels
- C. Format: PNG

CODE:

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

import pywt

# -----
# 1. Create a sample grayscale object image
# -----

img = np.zeros((512, 512), dtype=np.uint8)

# Draw a leaf-like object

points = np.array([
    [256, 70],
    [330, 130],
    [380, 220],
    [350, 330],
    [256, 430],
    [170, 330],
```



```
[130, 220],  
[180, 130]  
], np.int32)  
  
cv2.fillPoly(img, [points], 255)
```

```
# Add a central vein
```

```
cv2.line(  
    img,  
    (256, 90),  
    (256, 410),  
    120,  
    4  
)
```

```
# -----
```

```
# 2. Create multiple resolutions
```

```
# -----
```

```
img_128 = cv2.resize(  
    img,  
    (128, 128)  
)
```

```
img_256 = cv2.resize(  
    img,
```

```
(256, 256)

)

img_512 = img.copy()

# -----
# 3. Normalize images
# -----

img_128 = img_128.astype(float) / 255
img_256 = img_256.astype(float) / 255
img_512 = img_512.astype(float) / 255

# -----
# 4. Apply Haar Wavelet Transform
# -----

coeffs_128 = pywt.dwt2(
    img_128,
    'haar'
)

coeffs_256 = pywt.dwt2(
    img_256,
    'haar'
)
```

```

coeffs_512 = pywt.dwt2(
    img_512,
    'haar'
)

# Separate coefficients
LL128, (LH128, HL128, HH128) = coeffs_128
LL256, (LH256, HL256, HH256) = coeffs_256
LL512, (LH512, HL512, HH512) = coeffs_512

# -----
# 5. Calculate wavelet energy descriptors
# -----

def wavelet_energy(coefficients):

    LL, (LH, HL, HH) = coefficients

    energy_LL = np.sum(LL ** 2)
    energy_LH = np.sum(LH ** 2)
    energy_HL = np.sum(HL ** 2)
    energy_HH = np.sum(HH ** 2)

    total_energy = (
        energy_LL +
        energy_LH +

```

```
    energy_HL +  
    energy_HH  
)
```

```
return [  
    energy_LL,  
    energy_LH,  
    energy_HL,  
    energy_HH,  
    total_energy  
]
```

```
desc_128 = wavelet_energy(coeffs_128)  
desc_256 = wavelet_energy(coeffs_256)  
desc_512 = wavelet_energy(coeffs_512)
```

```
# -----
```

```
# 6. Print descriptors
```

```
# -----
```

```
print("=====")
```

```
print("MULTI-RESOLUTION WAVELET DESCRIPTORS")
```

```
print("=====")
```

```
print("\n128 x 128 Image")
```

```
print("LL Energy :", desc_128[0])
print("LH Energy :", desc_128[1])
print("HL Energy :", desc_128[2])
print("HH Energy :", desc_128[3])
print("Total   :", desc_128[4])
```

```
print("\n256 x 256 Image")
print("LL Energy :", desc_256[0])
print("LH Energy :", desc_256[1])
print("HL Energy :", desc_256[2])
print("HH Energy :", desc_256[3])
print("Total   :", desc_256[4])
```

```
print("\n512 x 512 Image")
print("LL Energy :", desc_512[0])
print("LH Energy :", desc_512[1])
print("HL Energy :", desc_512[2])
print("HH Energy :", desc_512[3])
print("Total   :", desc_512[4])
```

```
# -----
```

```
# 7. Display multi-resolution images
```

```
# -----
```

```
plt.figure(figsize=(12, 8))
```

```
plt.subplot(2, 3, 1)
plt.imshow(img_128, cmap='gray')
plt.title("128 × 128")
plt.axis("off")
```

```
plt.subplot(2, 3, 2)
plt.imshow(img_256, cmap='gray')
plt.title("256 × 256")
plt.axis("off")
```

```
plt.subplot(2, 3, 3)
plt.imshow(img_512, cmap='gray')
plt.title("512 × 512")
plt.axis("off")
```

```
# -----
```

```
# 8. Display LL wavelet coefficients
```

```
# -----
```

```
plt.subplot(2, 3, 4)
plt.imshow(LL128, cmap='gray')
plt.title("Wavelet LL - 128")
plt.axis("off")
```

```
plt.subplot(2, 3, 5)
plt.imshow(LL256, cmap='gray')
```

```
plt.title("Wavelet LL - 256")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 6)
```

```
plt.imshow(LL512, cmap='gray')
```

```
plt.title("Wavelet LL - 512")
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# -----
```

```
# 9. Wavelet coefficient visualization for 256 image
```

```
# -----
```

```
plt.figure(figsize=(10, 8))
```

```
plt.subplot(2, 2, 1)
```

```
plt.imshow(LL256, cmap='gray')
```

```
plt.title("LL - Approximation")
```

```
plt.axis("off")
```

```
plt.subplot(2, 2, 2)
```

```
plt.imshow(LH256, cmap='gray')
```

```
plt.title("LH - Horizontal Details")
```

```
plt.axis("off")
```

```
plt.subplot(2, 2, 3)

plt.imshow(HL256, cmap='gray')

plt.title("HL - Vertical Details")

plt.axis("off")
```

```
plt.subplot(2, 2, 4)

plt.imshow(HH256, cmap='gray')

plt.title("HH - Diagonal Details")

plt.axis("off")
```

```
plt.tight_layout()

plt.show()
```

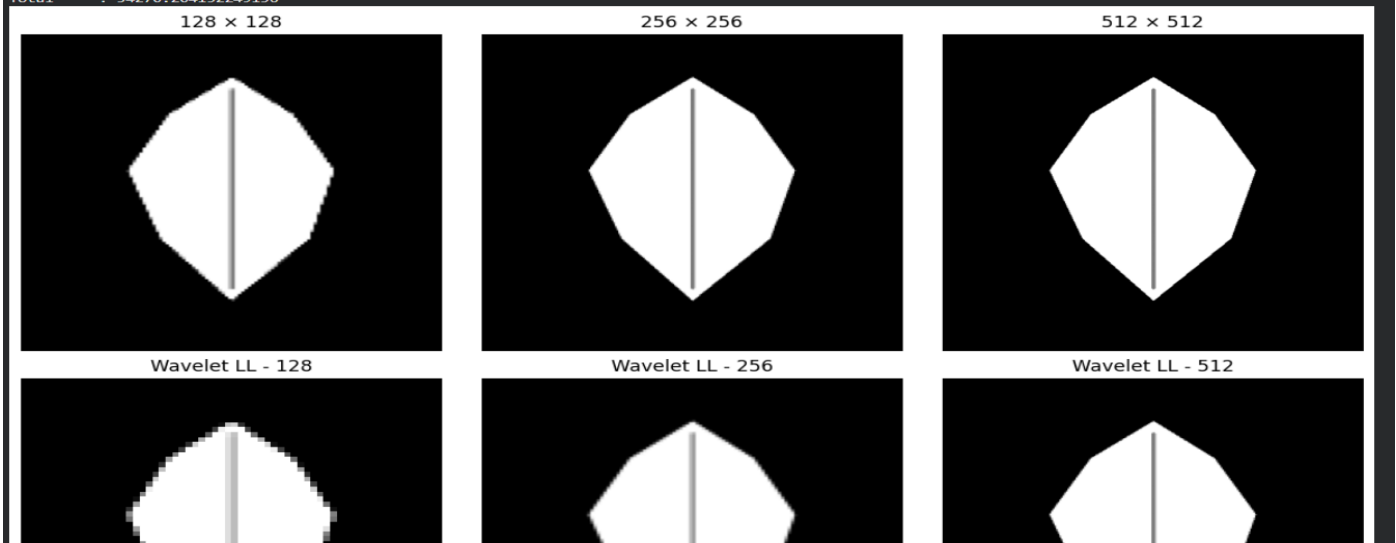
OUTPUT:

```
=====
MULTI-RESOLUTION WAVELET DESCRIPTORS
=====
```

```
128 x 128 Image
LL Energy : 3291.9807227989245
LH Energy : 12.641268742791238
HL Energy : 40.99542483660133
HH Energy : 9.885390234525186
Total     : 3355.5028066128425
```

```
256 x 256 Image
LL Energy : 13383.44722798924
LH Energy : 22.486858900422916
HL Energy : 75.31621683967708
HH Energy : 13.0562552864283
Total     : 13494.30655901577
```

```
512 x 512 Image
LL Energy : 53975.30882352943
LH Energy : 66.35034602076126
HL Energy : 169.19463667820077
HH Energy : 59.350346020761265
Total     : 54270.204152249156
```





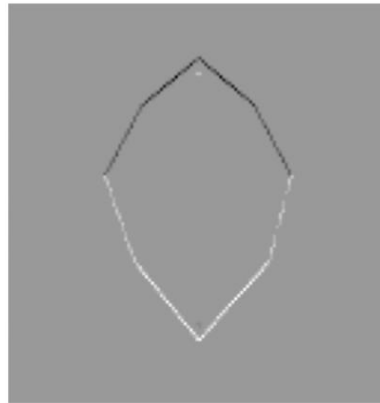
LL - Approximation



LH - Horizontal Details



HL - Vertical Details



HH - Diagonal Details

